

BABU BANARASI DAS UNIVERSITY

2026-27



SCHOOL OF COMPUTER APPLICATIONS

NoSQL PROJECT

ON

STUDENT MANAGEMENT SYSTEM

USING MongoDB

Submitted to:

Mr. Harendra Singh

Department: BCA (DS & AI)

Date: 28th September

Submitted by:

Akshit Yadav

Batch:BCADS21

Uni_Roll_no.:

1250258063

ACKNOWLEDGEMENT

I sincerely express my gratitude to my teacher **Mr. Harendra Singh** for giving me the opportunity to work on this NoSQL project titled "**Student Management System Using MongoDB**".

This project helped me understand the basic concepts of NoSQL databases and learn how MongoDB is used to **create, retrieve, update, Sorting and delete** data. I also learned how different query operators such as **greater than, less than, equal to, AND, and OR** are used in MongoDB.

I am thankful to my teacher for the guidance and support provided throughout the completion of this project.

INTRODUCTION

MongoDB is a popular NoSQL database management system that stores data in the form of **documents**. Unlike traditional relational databases, MongoDB does not require data to be stored in fixed rows and columns.

In this project, a Student Management System has been created using MongoDB. The database contains information about **50 students**, including their **roll number, name, age, marks, and city**.

Different MongoDB operations have been performed on the student data, such as **inserting, displaying, updating, deleting, sorting, counting, and searching** records. Comparison operators such as **\$eq, \$gt, \$lt, \$gte, and \$lte** have also been demonstrated.

OBJECTIVES

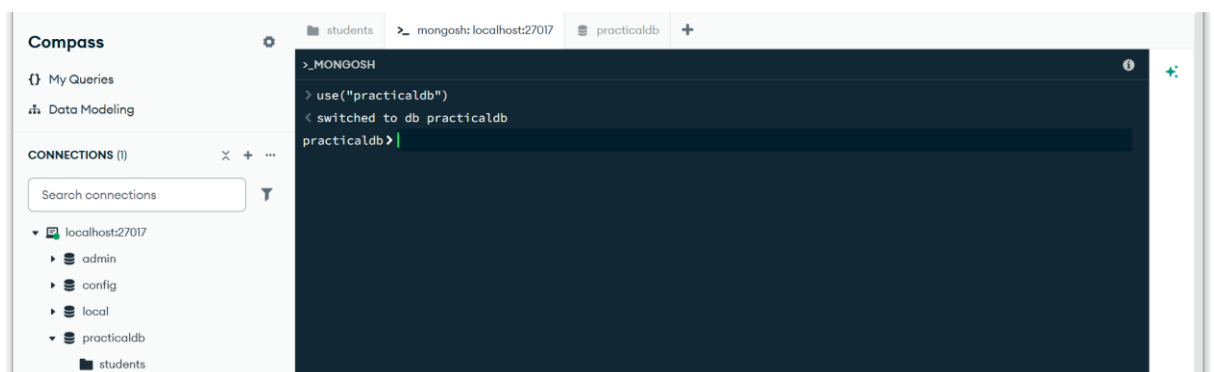
- To understand the concept of NoSQL databases.
- To learn the basic features of MongoDB.
- To **create** a database and collection.
- To insert **50 student** records.
- To perform **CRUD** operations.
- To use comparison operators.
- To perform sorting and filtering.
- To understand **AND** and **OR** operations.

1. CREATE DATABASE

MongoDB Query :

use ("practicaldb")

Explanation : Creates or Switches to the practicaldb database.

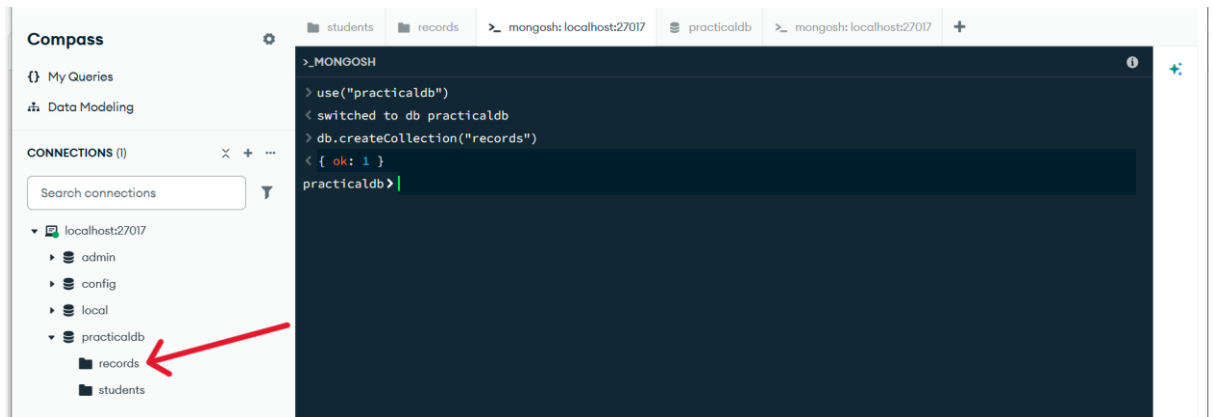


2. CREATE COLLECTION

MongoDB Query :

```
db.createCollection("records")
```

Explanation : Creates a new collection into a database.

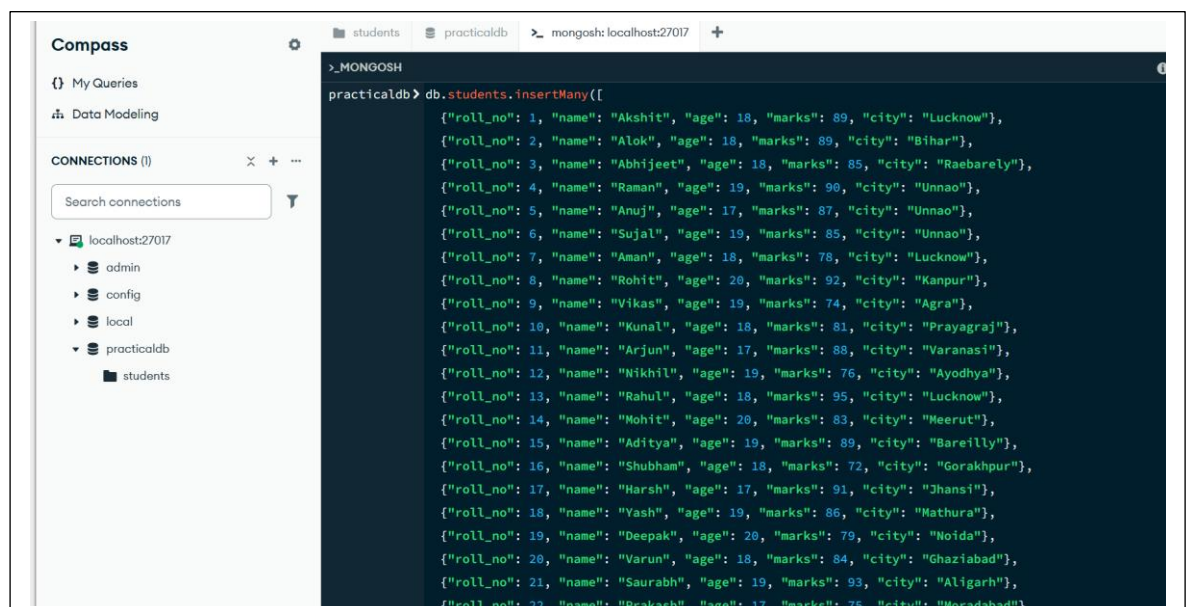


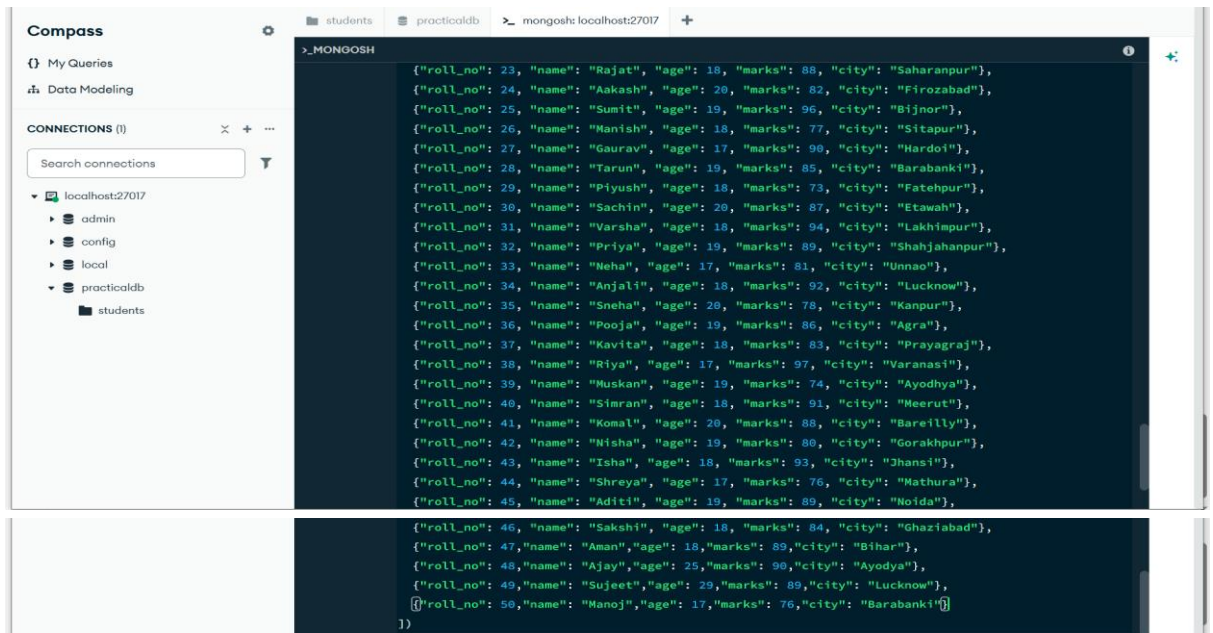
3. INSERT 50 STUDENTS DATA

MongoDB Query :

```
db.students.insertMany({
  50 students data.....
})
```

Explanation : Insert 50 Students documents(data) at once into the students collection.



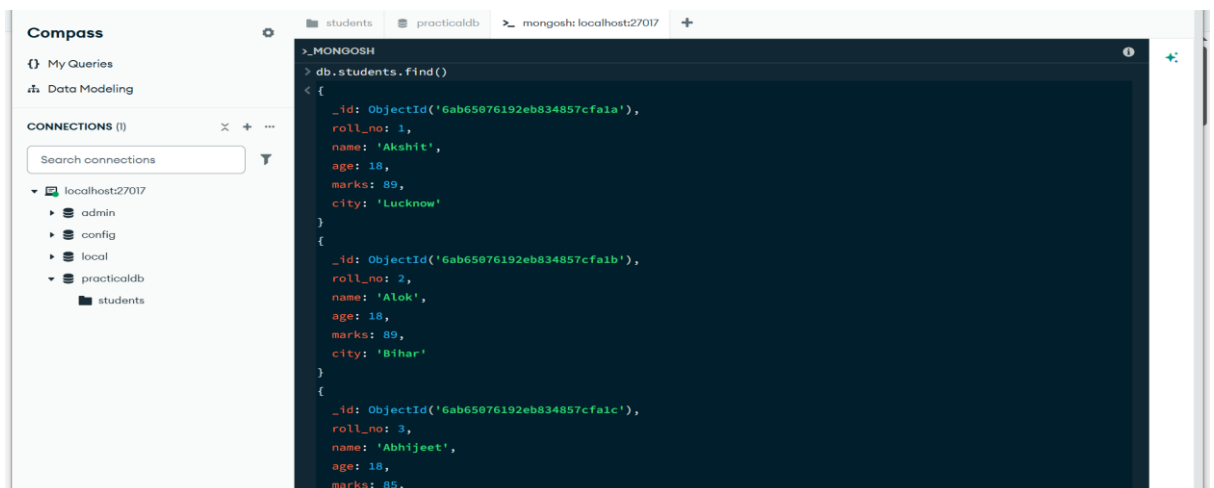


4. DISPLAY ALL DOCUMENTS

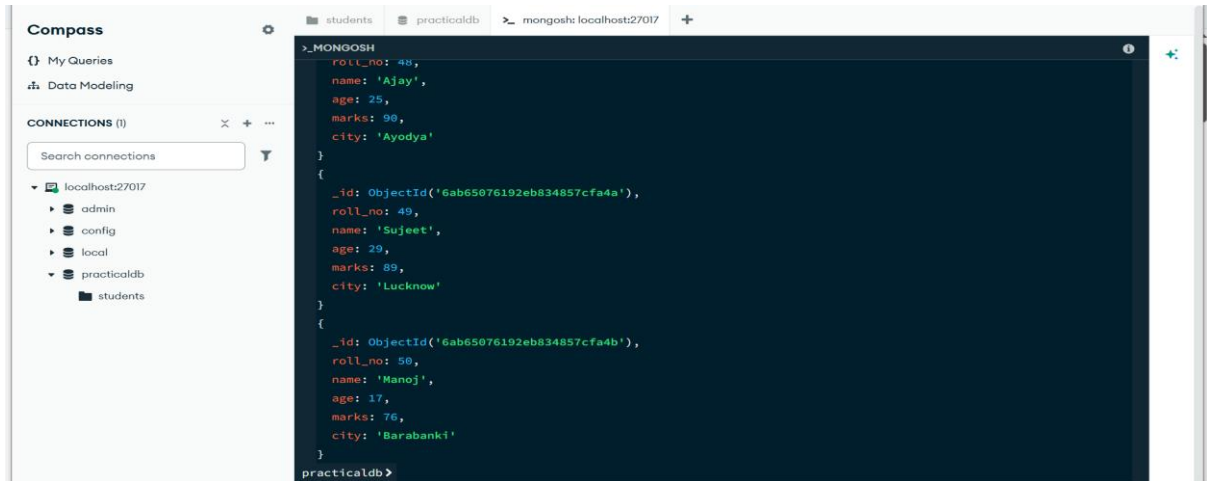
MongoDB Query :

Db.students.find()

Explanation : Retrieves all documents from the students collection.



-
-
-
-

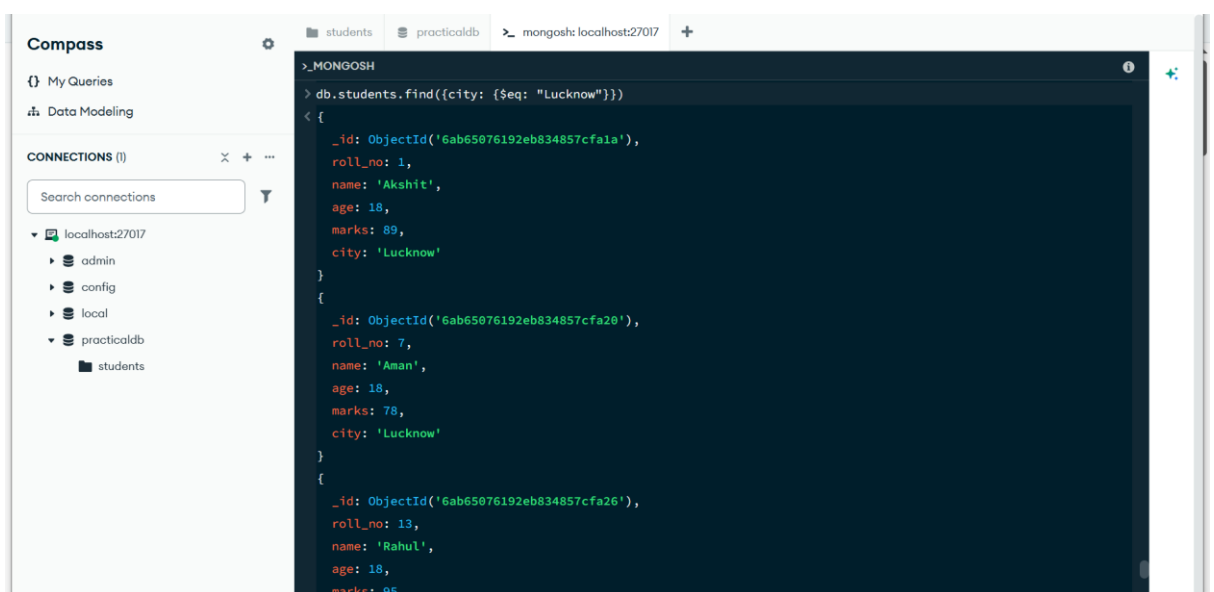


5. EQUAL TO - \$eq

Mongodb Query :

`db.students.find({city: {$eq: "Lucknow"}})`

Explanation : Finds that documents whose city is Lucknow.

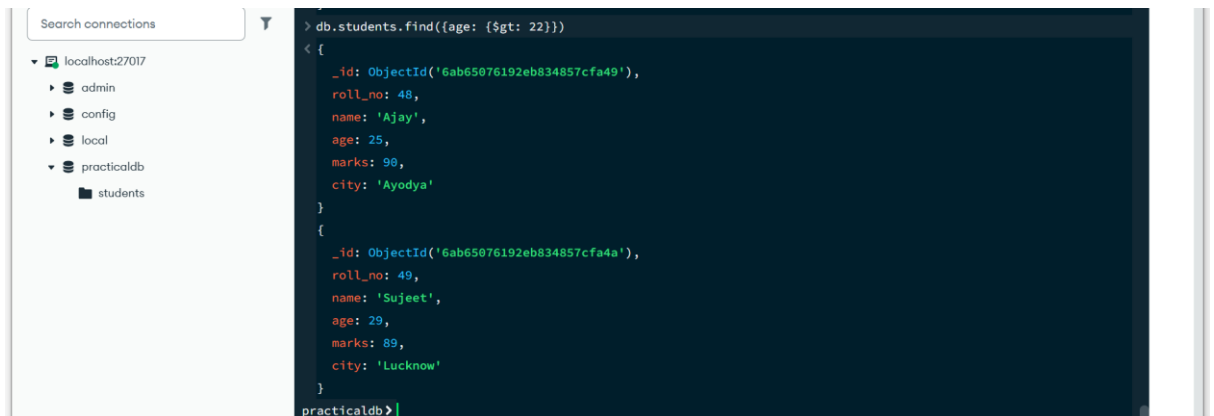


6. GREATER THAN - \$gt

MongoDB :

`db.students.find({age: {$gt: 22}})`

Explanation : Finds those documents whose age is greater than 22.



The screenshot shows the MongoDB Compass interface. On the left, the 'connections' panel shows a connection to 'localhost:27017' with a database named 'practicaldb' and a collection named 'students'. The main panel displays the query `db.students.find({age: {$gt: 22}})` and its results. The results are two documents:

```
{
  "_id": ObjectId('6ab65076192eb834857cfa49'),
  "roll_no": 48,
  "name": 'Ajay',
  "age": 25,
  "marks": 90,
  "city": 'Ayodhya'
}
```

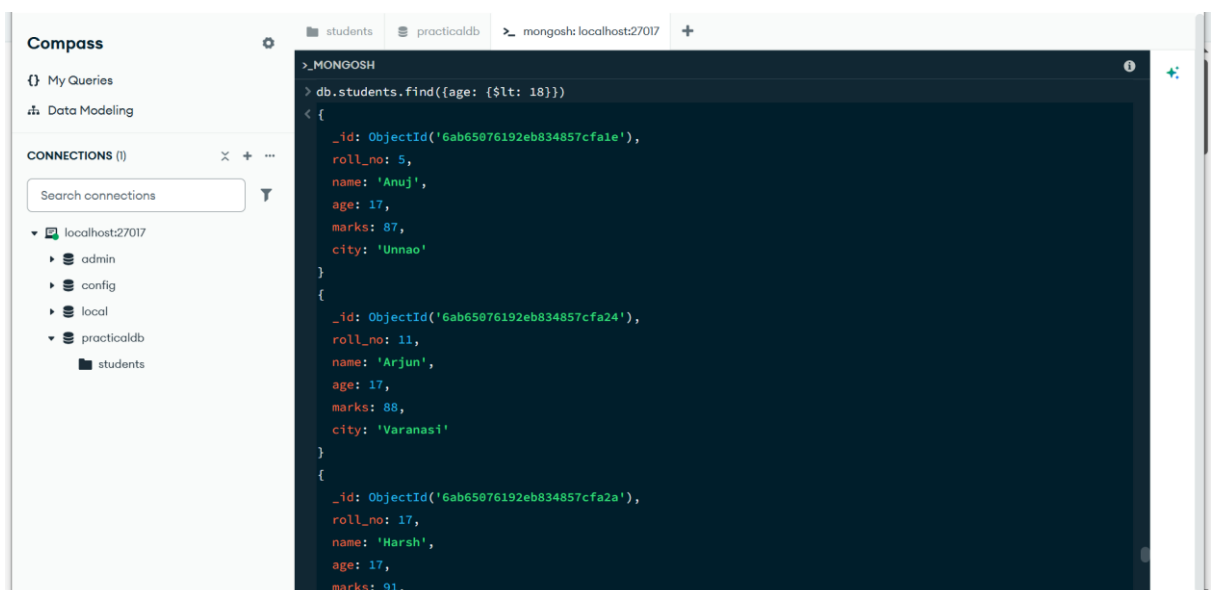
```
{
  "_id": ObjectId('6ab65076192eb834857cfa4a'),
  "roll_no": 49,
  "name": 'Sujeet',
  "age": 29,
  "marks": 89,
  "city": 'Lucknow'
}
```

7. LESS THAN - \$lt

MongoDB Query :

`db.students.find({age: {$lt: 18}})`

Explanation : Finds those documents whose age is less than 18.



The screenshot shows the MongoDB Compass interface. On the left, the 'connections' panel shows a connection to 'localhost:27017' with a database named 'practicaldb' and a collection named 'students'. The main panel displays the query `db.students.find({age: {$lt: 18}})` and its results. The results are three documents:

```
{
  "_id": ObjectId('6ab65076192eb834857cfa1e'),
  "roll_no": 5,
  "name": 'Anuj',
  "age": 17,
  "marks": 87,
  "city": 'Unnao'
}
```

```
{
  "_id": ObjectId('6ab65076192eb834857cfa24'),
  "roll_no": 11,
  "name": 'Arjun',
  "age": 17,
  "marks": 88,
  "city": 'Varanasi'
}
```

```
{
  "_id": ObjectId('6ab65076192eb834857cfa2a'),
  "roll_no": 17,
  "name": 'Harsh',
  "age": 17,
  "marks": 91
}
```

8. GREATER THAN OR EQUAL TO - \$gte

MongoDB Query :

```
db.students.find({marks: {$gte: 96}})
```

Explanation : Finds documents whose marks is 96 or above.



The screenshot shows the MongoDB CLI interface. On the left, a sidebar lists connections: localhost:27017, admin, config, local, and practicaldb (selected). Under practicaldb, the 'students' collection is listed. The main terminal area shows the command `> db.students.find({marks: {$gte: 96}})` and its output, which is a JSON array of two documents. The first document is for a student named 'Sumit' with a roll number of 25, age 19, marks 96, and city 'Bijnor'. The second document is for a student named 'Riya' with a roll number of 38, age 17, marks 97, and city 'Varanasi'.

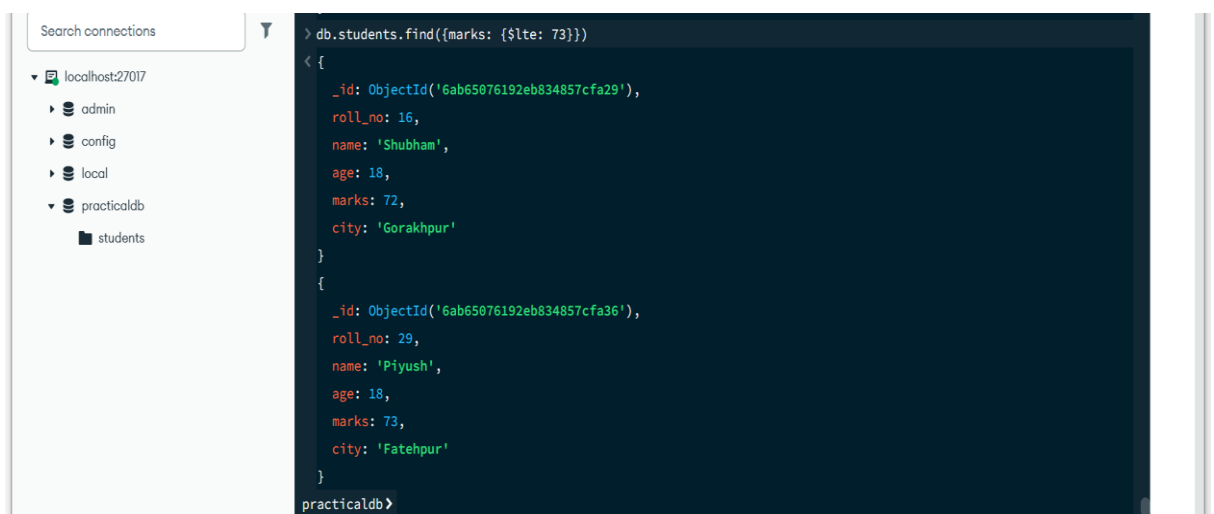
```
> db.students.find({marks: {$gte: 96}})
< {
  _id: ObjectId('6ab65076192eb834857cfa32'),
  roll_no: 25,
  name: 'Sumit',
  age: 19,
  marks: 96,
  city: 'Bijnor'
}
{
  _id: ObjectId('6ab65076192eb834857cfa3f'),
  roll_no: 38,
  name: 'Riya',
  age: 17,
  marks: 97,
  city: 'Varanasi'
}
practicaldb>
```

9. LESS THAN OR EQUAL TO - \$lte

MongoDB Query :

```
db.students.find({marks: {$lte: 73}})
```

Explanation : Finds documents whose marks 73 or below.



The screenshot shows the MongoDB CLI interface. On the left, a sidebar lists connections: localhost:27017, admin, config, local, and practicaldb (selected). Under practicaldb, the 'students' collection is listed. The main terminal area shows the command `> db.students.find({marks: {$lte: 73}})` and its output, which is a JSON array of two documents. The first document is for a student named 'Shubham' with a roll number of 16, age 18, marks 72, and city 'Gorakhpur'. The second document is for a student named 'Piyush' with a roll number of 29, age 18, marks 73, and city 'Fatehpur'.

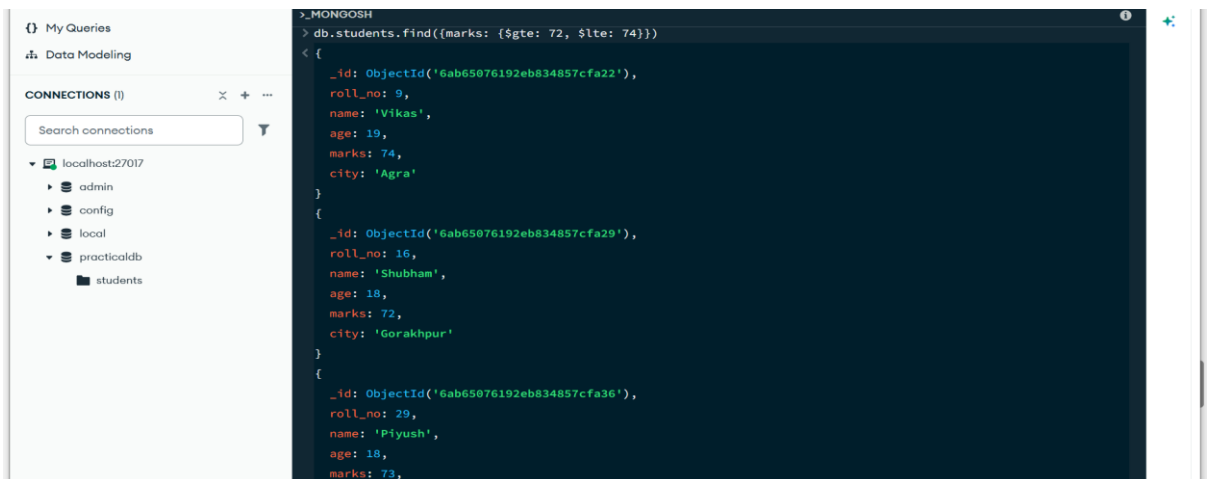
```
> db.students.find({marks: {$lte: 73}})
< {
  _id: ObjectId('6ab65076192eb834857cfa29'),
  roll_no: 16,
  name: 'Shubham',
  age: 18,
  marks: 72,
  city: 'Gorakhpur'
}
{
  _id: ObjectId('6ab65076192eb834857cfa36'),
  roll_no: 29,
  name: 'Piyush',
  age: 18,
  marks: 73,
  city: 'Fatehpur'
}
practicaldb>
```

10. USE \$gte AND \$lte TOGETHER

MongoDB Query :

```
db.students.find({marks: {$gte: 72, $lte: 74}})
```

Explanation : Finds document whose marks have between 72 to 74 including 72 and 74.



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel lists 'localhost:27017' with a tree view showing 'practicaldb' > 'students'. The main panel displays the query `db.students.find({marks: {$gte: 72, $lte: 74}})` and its results. The results are three documents:

```
< {
  _id: ObjectId('6ab65076192eb834857cfa22'),
  roll_no: 9,
  name: 'Vikas',
  age: 19,
  marks: 74,
  city: 'Agra'
}
{
  _id: ObjectId('6ab65076192eb834857cfa29'),
  roll_no: 16,
  name: 'Shubham',
  age: 18,
  marks: 72,
  city: 'Gorakhpur'
}
{
  _id: ObjectId('6ab65076192eb834857cfa36'),
  roll_no: 29,
  name: 'Piyush',
  age: 18,
  marks: 73,
```

11. UPDATE OPERATION

MongoDB Query :

```
db.students.updateOne({name: "Akshit"}, {$set: {city: "Unnao"}})
```

Explanation : Update the city "Lucknow" to "Unnao" with name Akshit.



The screenshot shows the MongoDB Compass interface. The 'CONNECTIONS' panel shows 'localhost:27017' with a tree view showing 'practicaldb' > 'records' > 'students'. The main panel displays the query `use("practicaldb")` and `db.students.updateOne({name: "Akshit"}, {$set: {city: "Unnao"}})`. The results show the update operation details:

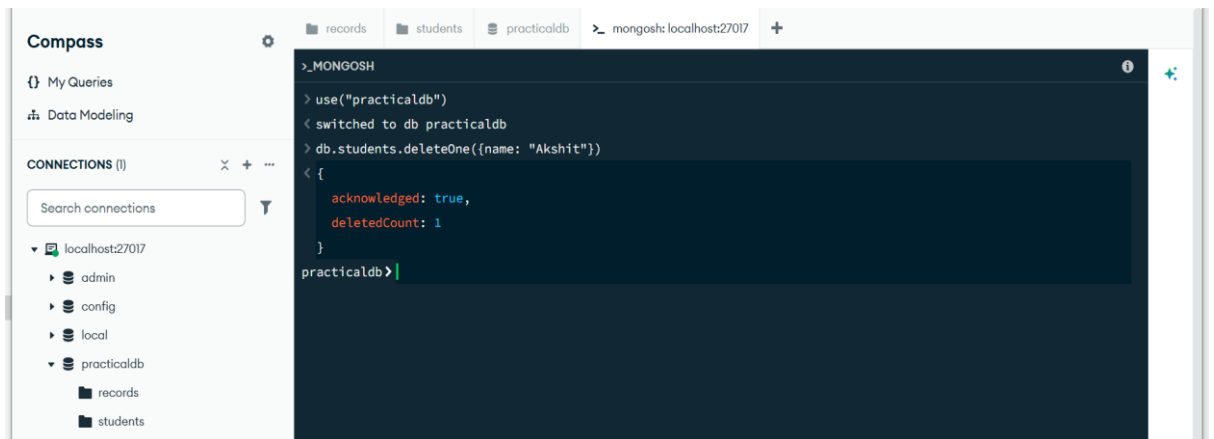
```
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
practicaldb>
```

12. DELETE ONE OPERATION

MongoDB Query :

```
db.students.deleteOne({name: "Akshit"})
```

Explanation : Delete that one document whose name is Akshit.



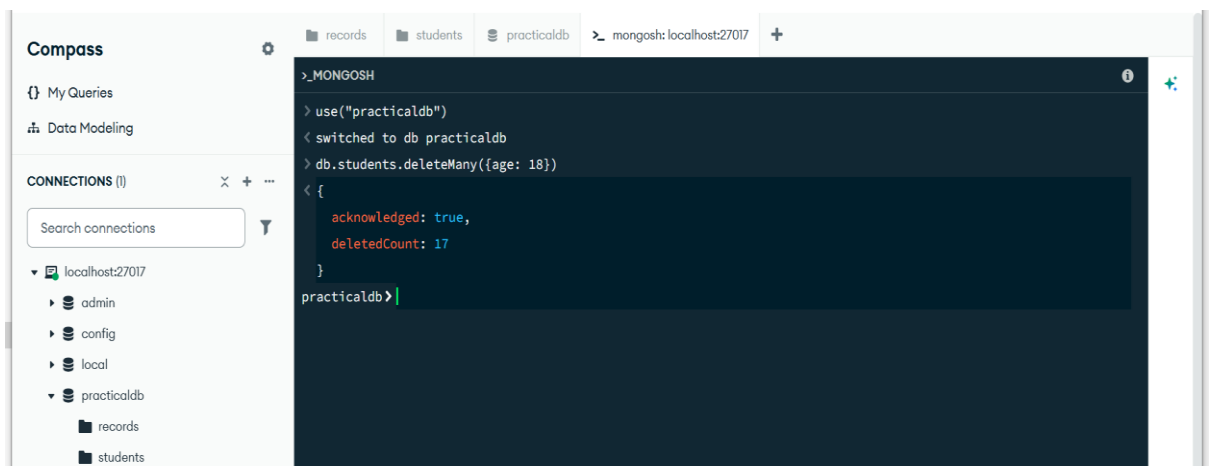
13. DELETE MANY OPERATION

MongoDB Query :

```
db.students.deleteMany({age: 18})
```

Exception : Delete all documents whose age is 18.

(There are 17 documents in which age is 18.)

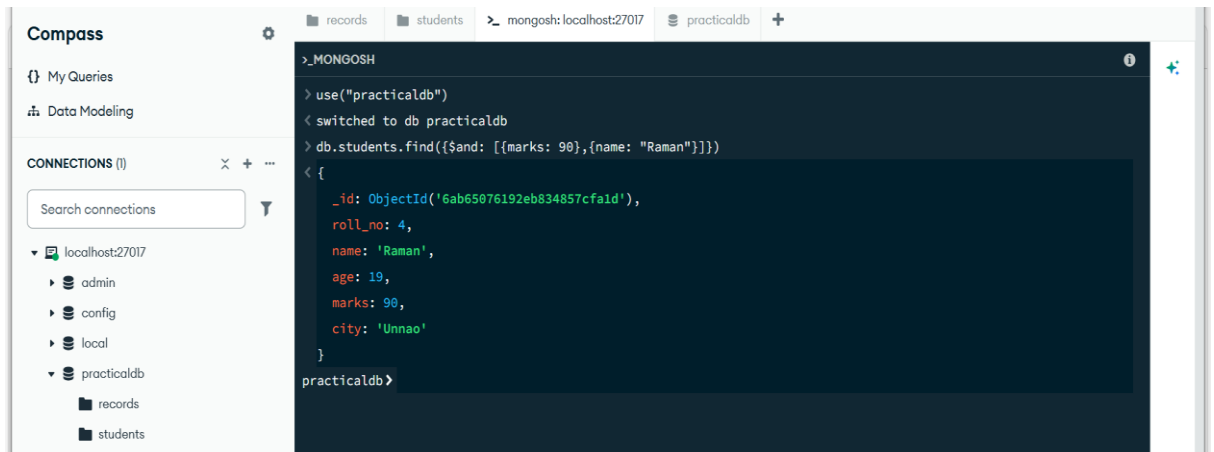


14. AND - \$and

MongoDB Query :

```
db.students.find({$and: [{marks: 90}, {name: "Raman"}]})
```

Explanation : Finds documents which satisfying both conditions.

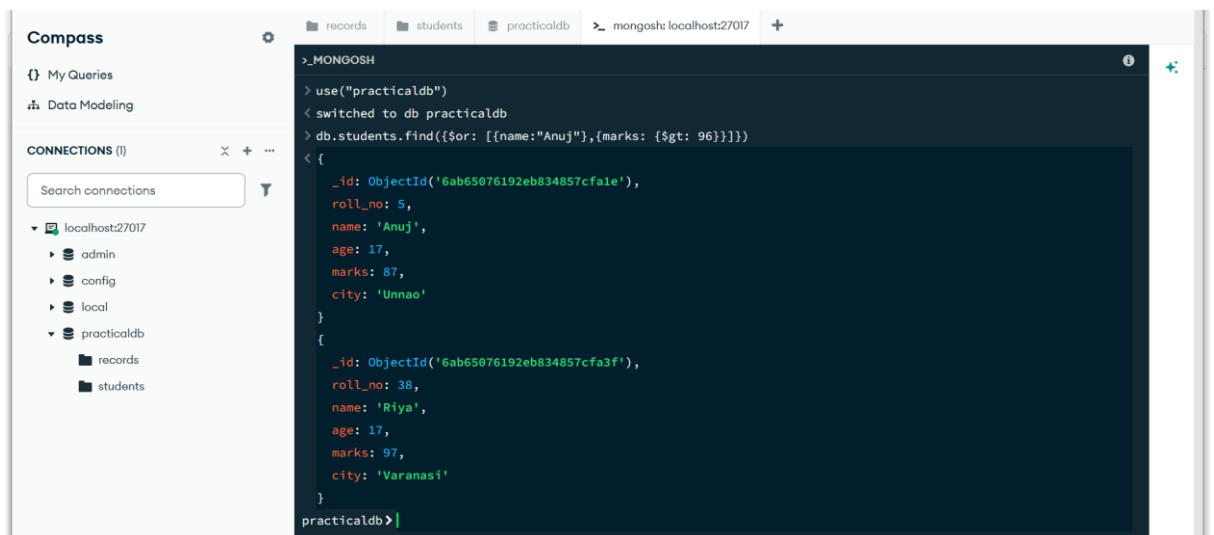


15. OR - \$or

MongoDB Query :

```
db.students.find({$or: [{name: "Anuj"}, {marks: {$gt: 96}}]})
```

Explanation : Find documents which satisfying either one or both conditions.



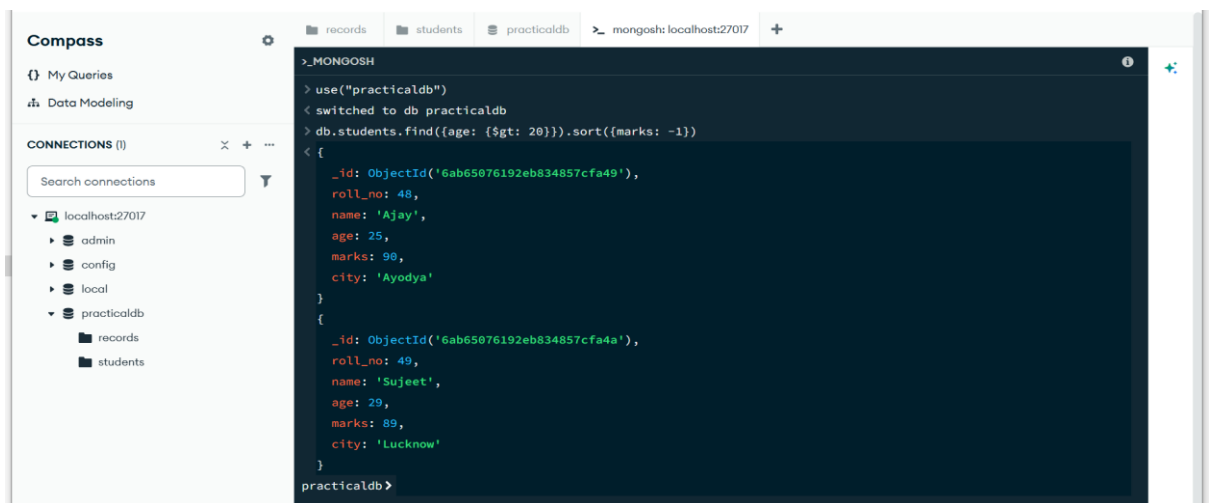
16. SORT OPERATION

A. IN DESCENDING ORDER

MongoDB Query :

```
db.students.find({age: {$gt: 20}}).sort({marks: -1})
```

Explanation : Sort their marks from highest to lowest whose age is above 20. Use -1 for Descending Order.



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel shows a connection to 'localhost:27017' with a database named 'practicaldb'. The main panel displays the MongoDB shell output for the query `db.students.find({age: {$gt: 20}}).sort({marks: -1})`. The results are sorted by marks in descending order, showing two documents: one for 'Ajay' with marks 98 and one for 'Sujeet' with marks 89.

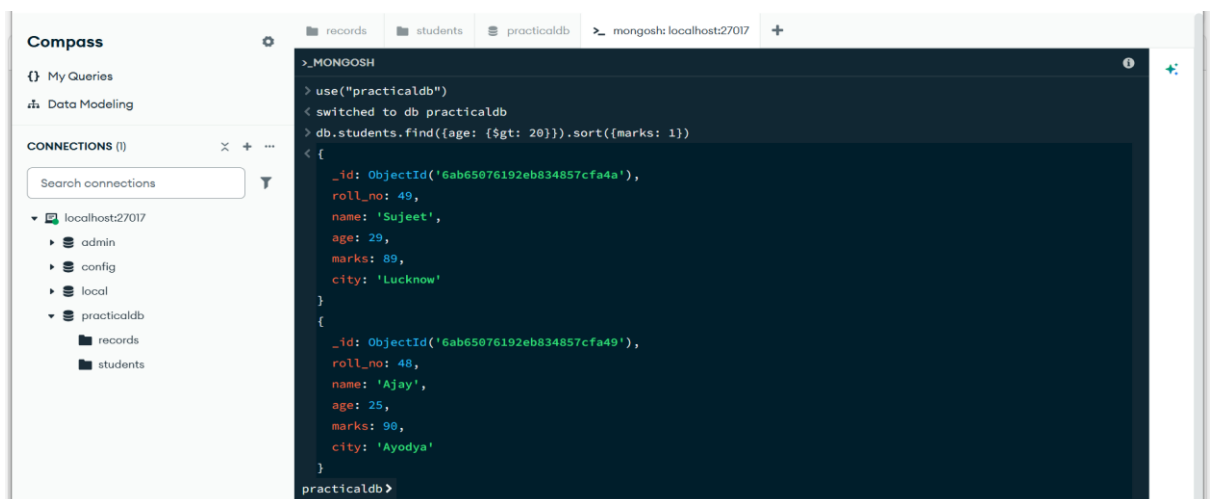
```
>_MONGOSH
> use("practicaldb")
< switched to db practicaldb
> db.students.find({age: {$gt: 20}}).sort({marks: -1})
< [
  {
    _id: ObjectId('6ab65076192eb834857cfa49'),
    roll_no: 48,
    name: 'Ajay',
    age: 25,
    marks: 98,
    city: 'Ayodhya'
  },
  {
    _id: ObjectId('6ab65076192eb834857cfa4a'),
    roll_no: 49,
    name: 'Sujeet',
    age: 29,
    marks: 89,
    city: 'Lucknow'
  }
]
practicaldb>
```

B. IN ASCENDING ORDER

MongoDB Query :

```
db.students.find({age: {$gt: 20}}).sort({marks: 1})
```

Explanation : Sort their marks from lowest to highest whose age is above 20. Use 1 for Ascending Order.



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel shows a connection to 'localhost:27017' with a database named 'practicaldb'. The main panel displays the MongoDB shell output for the query `db.students.find({age: {$gt: 20}}).sort({marks: 1})`. The results are sorted by marks in ascending order, showing two documents: one for 'Sujeet' with marks 89 and one for 'Ajay' with marks 98.

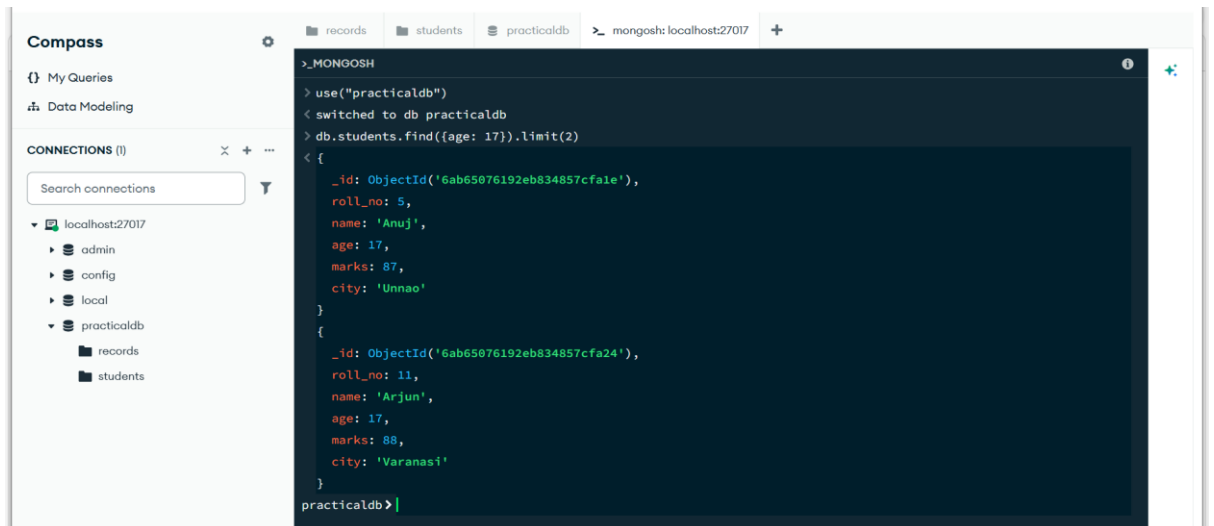
```
>_MONGOSH
> use("practicaldb")
< switched to db practicaldb
> db.students.find({age: {$gt: 20}}).sort({marks: 1})
< [
  {
    _id: ObjectId('6ab65076192eb834857cfa4a'),
    roll_no: 49,
    name: 'Sujeet',
    age: 29,
    marks: 89,
    city: 'Lucknow'
  },
  {
    _id: ObjectId('6ab65076192eb834857cfa49'),
    roll_no: 48,
    name: 'Ajay',
    age: 25,
    marks: 98,
    city: 'Ayodhya'
  }
]
practicaldb>
```

17. LIMIT OPERATION

MongoDB Query :

```
db.students.find({age: 17}).limit(2)
```

Explanation : Display only first two documents whose age is 17.

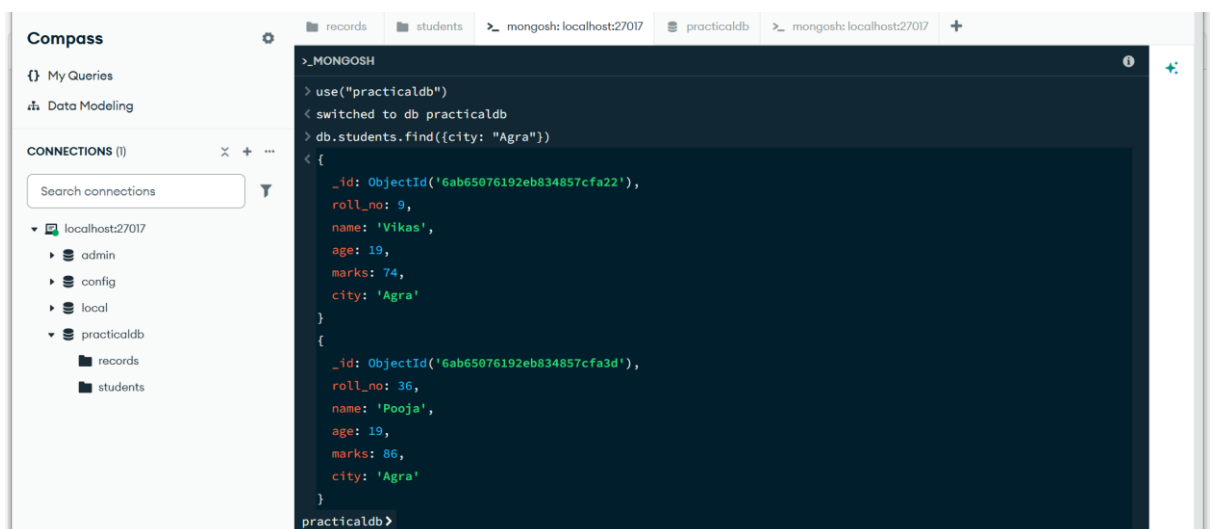


18. SEARCH DOCUMENT THROUGH SOME DETAIL (BY CITY)

MongoDB Query :

```
db.students.find({city: "Agra"})
```

Explanation : Finds all documents whose city is Agra.

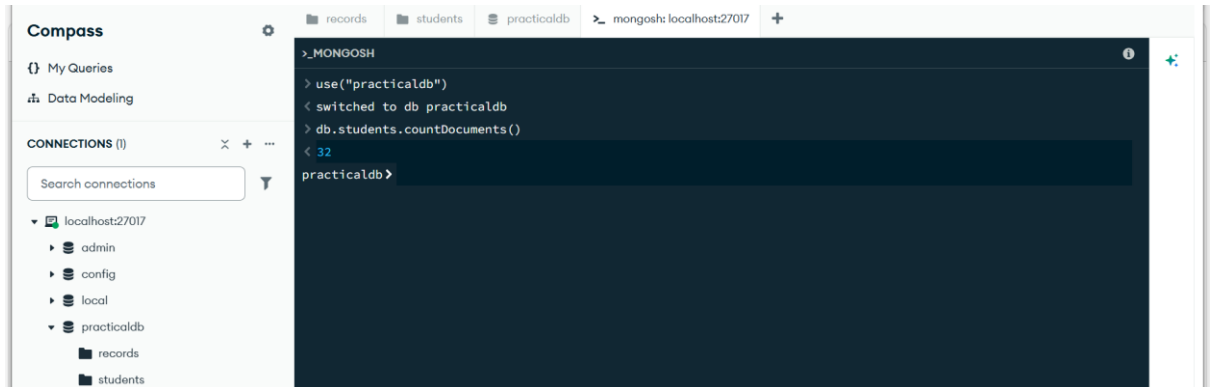


19. COUNT DOCUMENTS

MongoDB Query :

```
db.students.countDocuments()
```

Explanation : Counts documents currently present in the collection (performing after the delete function).



MongoDB OPERATORS

OPERATOR	MEANING	EXAMPLE
\$eq	Equal to	marks: {\$eq:88}
\$gt	Greater than	marks: {\$gt:85}
\$lt	Less than	marks: {\$lt:60}
\$gte	Greater than or equal to	marks: {\$gte:90}
\$lte	Less than or equal to	marks: {\$lte:70}
\$and	All conditions true	\$and:[...]
\$or	At least one condition true	\$or:[...]

CONCLUSION

This project provided practical knowledge of NoSQL databases using **MongoDB**. A Student Management System containing student information was created and different MongoDB operations were performed on the data.

Through this project, I learned how to **insert**, **retrieve**, **update**, and **delete** documents. I also learned how to Ascending order & descending order data, comparison operators such as **\$eq**, **\$gt**, **\$lt**, **\$gte**, and **\$lte** work, along with logical operators such as **\$and** and **\$or**.

Overall, this project helped me understand how MongoDB can be used to manage and query student data efficiently.